TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

Chương 5
TẬP LỆNH ARM
(Phần 1)



1

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
HCM·UTE
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

NỘI DUNG

- Giới thiệu chung về đặc điểm của ARM
- Các thanh ghi trong ARM
- Chu kỳ nạp – giải mã – thi hành lệnh
- Trật tự byte
- Mô hình lập trình cho ARM
- Tập lệnh thumb
- Định dạng tổng quát của lệnh
- Giới thiệu mã hóa tập lệnh
- Các loại lệnh của CPU
- Nhóm lệnh di chuyển dữ liệu
- Các lệnh số học

2

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Characteristic of ARM Architecture

- CISC vs RISC
  - CISC: MULT 2:3, 5:2
  - RISC:
    - MOV A, 2:3
    - MOV B, 5:2
    - MUL A, B
- RISC enables data processing primarily at the **register level**, providing several advantages:
  - Low Power Consumption
    - Optimized for energy efficiency.
    - Widely used in mobile phones, tablets, and embedded systems.
  - High Performance per Watt

3

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Characteristic of ARM Architecture

- Load/Store Architecture
  - **Operands of arithmetic and logical instructions** (such as addition, subtraction, etc.) are stored in and processed within **registers**.
  - **Only load and store instructions** access memory:
    - **Load** transfers data from memory to a register.
    - **Store** transfers data from a register back to memory.
  - **CISC processors** can perform operations directly on memory locations; however, this generally results in **slower execution speed** compared to register-based processing.

4

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Registers in the Cortex-M3

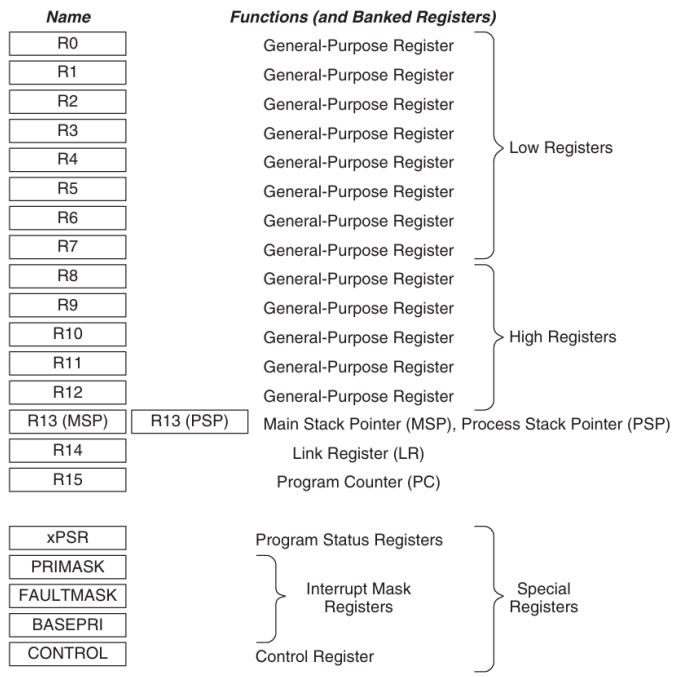


Figure 3.1 Registers in the Cortex-M3

5

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ARM Cortex M3 Registers

- R0 – R7: Low Registers
  - All are 32-bit wide
  - Can be accessed by all 16-bit THUMB instruction
  - Can be accessed by all 32-bit THUMB 2 instruction
  - Reset value is unpredictable
- R8 – R12: High Registers
  - All are 32-bit wide
  - Cannot be accessed by all 16-bit THUMB instruction
  - Can be accessed by all 32-bit THUMB instruction
  - Reset value is inpredictable

6

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ARM Cortex M3 Registers

- R13: Stack Pointer
  - There are **two stack pointers** in ARM Cortex-M processors:
    - **MSP (Main Stack Pointer)**
    - **PSP (Process Stack Pointer)**
  - **MSP** is the default stack pointer and is used by:
    - The OS kernel
    - Exception handlers
    - Application code that requires **privileged access**
  - **PSP** is used by:
    - Base-level (non-privileged) application code
    - When the processor is **not** running an exception handler
  - When the register name **R13** appears in code, current stack pointer is accessed
  - It is **not required** to use both stack pointers. Simple applications can rely solely on the **MSP**.
  - Stack pointers are used for stack memory operations such as:
    - **PUSH**
    - **POP**

7

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

BASIC STACK OPERATION

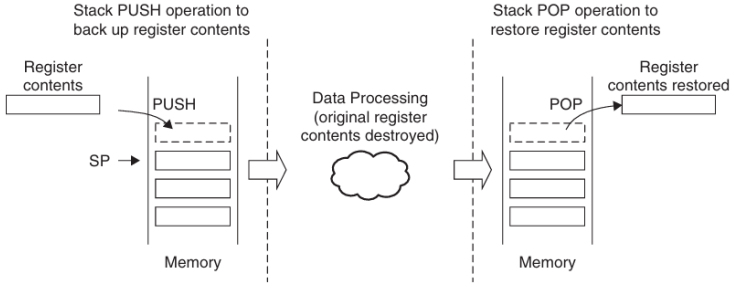


Figure 3.2 Basic Concept of Stack Memory

- When doing PUSH and POP operations, the pointer register, commonly called stack pointer, is adjusted automatically to prevent next stack operations from corrupting previous stacked data

8

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

BASIC STACK OPERATION

- PUSH and POP instructions are used for accessing stack memory, Usually in the begining and end of a subroutine to save and restore the register values
- PUSH and POP syntax

```
PUSH        {R0}   ; R13=R13-4, then Memory[R13] = R0
POP         {R0}   ; R0 = Memory[R13], then R13 = R13+4
```

- When new data is pushed to stack, stack pointer decrements
- Multiple registers can be pushed to stack and poped from stack.
  - PUSH {R0-R7, R12, R14}
  - POP {R0-R7, R12, R14}
- Instead of using R13, SP can be used
- Register PUSH and POP operations are world alligned

9

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

BASIC STACK OPERATION - EXAMPLE

```
MOV  R0, #25
PUSH {R0}
MOV  R0, #100
POP  {R0}
```

Với SP = 0x20001000, kết quả SP, R0 sau thực hiện chương trình là bao nhiêu?

10

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ARM Cortex M3 Register

- R14: Link Register
  - LR is used to store the return program counter (PC) when a subroutine or function is called
  - Example

```
main    ; Main program
        …
        BL   function1 ; Call function1 using Branch with Link
                       ; instruction.
                       ; PC = function1 and
                       ; LR = the next instruction in main
        …
function1
        …              ; Program code for function 1
        BX   LR        ; Return
```

13

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ARM Cortex M3 Register - Example

```
        MOV R0,#5
        BL function1
        MOV R1,#20
stop
        B stop
function1
        ADD R0,R0,#10
        BX LR
```

R0 = ? R1 = ?

4.14

14

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ARM Cortex M3 Registers

- R15: Program Counter
  - R15 or PC
  - Because of the pipelined nature of the Cortex-M3 Processor, when you read this register, you will find that the value is different than the location of the executing instruction, normally by 4.
  - For example:

```
0x1000 :    MOV    R0, PC    ; R0 = 0x1004
```

  - LSB of this bit is always 0

19

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ARM Cortex M3 Registers - Example

0x1000 MOV R0, PC

0x1002 ADD R1, R1, #1

0x1004 SUB R2, R2, #1

PC = 0x1004

20

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

SPECIAL REGISTERS

- Special Registers in Cortex M3 processor
  - Program Status registers (PSRs)
  - Interrupt Mask registers (PRIMASK, FAULTMASK, and BASEPRI)
  - Control register (CONTROL)
- Special registers can only be accessed via MSR and MRS instructions

```
MRS    <reg>, <special_reg>; Read special register
MSR    <special_reg>, <reg>; write to special register
```


HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

PROGRAM STATUS REGISTER

- Application Program Status register (APSR), *Bit 31–27 of xPSR*
- Interrupt Program Status register (IPSR), *Bit 8–0 of xPSR*
- Execution Program Status register (EPSR), *Bit 26–10 of xPSR*

	31	*30*	*29*	*28*	*27*	*26:25*	*24*	*23:20*	*19:16*	*15:10*	*9*	*8*	*7*	*6*	*5*	*4:0*
APSR	N	Z	C	V	Q											
IPSR												Exception Number				
EPSR						ICI/IT	T			ICI/IT						

Figure 3.3 Program Status Registers (PSRs) in the Cortex-M3

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Accessing PSR.. Examples

- You can read the program status registers using the MRS instruction. You can also change the APSR using the MSR instruction, but EPSR and IPSR are read-only. For example:

```
MRS     r0, APSR     ; Read Flag state into R0
MRS     r0, IPSR     ; Read Exception/Interrupt state
MRS     r0, EPSR     ; Read Execution state
MSR     APSR, r0     ; Write Flag state
```

- In ARM assembler, when accessing xPSR (all three program status registers as one), the symbol PSR is used:

```
MRS     r0, PSR     ; Read the combined program status word
MSR     PSR, r0     ; Write combined program state word
```


25

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Bit fields of PSR

Table 3.1 Bit Fields in Cortex-M3 Program Status Registers

Bit	Description
N	Negative
Z	Zero
C	Carry/borrow
V	Overflow
Q	Sticky saturation flag
ICI/IT	Interrupt-Continuable Instruction (ICI) bits, IF-THEN instruction status bit
T	Thumb state, always 1; trying to clear this bit will cause a fault exception
Exception Number	Indicates which exception the processor is handling

26

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Bit fields of PSR - Example

MOV R0,#3
MOV R1,#5
SUBS R2,R0,R1

N = ?, Z = ?

MOV R0,#5
MOV R1,#5
SUBS R2,R0,R1
N = ?, Z = ?

27

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Bit fields of PSR - Example

- 11111111 + 00000001 = ?
- 01111111 + 00000001 = ?

MOV R1,#5
MOV R2,#5
SUBS R0,R1,R2

R0 = ? Z = ? N = ?

28

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Cortex-M3 Interrupt Mask Registers

Table 3.2 Cortex-M3 Interrupt Mask Registers

Register Name	Description
PRIMASK	A 1-bit register. When this is set, it allows NMI and the hard fault exception; all other interrupts and exceptions are masked. The default value is 0, which means that no masking is set.
FAULTMASK	A 1-bit register. When this is set, it allows only the NMI, and all interrupts and fault handling exceptions are disabled. The default value is 0, which means that no masking is set.
BASEPRI	A register of up to 9 bits (depending on the bit width implemented for priority level). It defines the masking priority level. When this is set, it disables all interrupts of the same or lower level (larger priority value). Higher-priority interrupts can still be allowed. If this is set to 0, the masking function is disabled (this is the default).

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Cortex-M3 Interrupt Mask Registers - Example

- To access the PRIMASK, FAULTMASK, and BASEPRI registers, the MRS and MSR instructions are used. For example:

```
MRS     r0, BASEPRI    ; Read BASEPRI register into R0
MRS     r0, PRIMASK    ; Read PRIMASK register into R0
MRS     r0, FAULTMASK  ; Read FAULTMASK register into R0
MSR     BASEPRI, r0    ; Write R0 into BASEPRI register
MSR     PRIMASK, r0    ; Write R0 into PRIMASK register
MSR     FAULTMASK, r0  ; Write R0 into FAULTMASK register
```

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

CONTROL REGISTER

- The Control register is used to define the privilege level and the stack pointer selection. This register has two bits. These bits are write-only when the core is in thread mode and privileged. In user state or handler mode, writing to these bits is not allowed.
- Another way to change this bit is by modifying bit 2 of the link register (LR) during exception return.

Table 3.3 Cortex-M3 CONTROL Register

Bit	Function
CONTROL[1]	Stack status: 1 = Alternate stack is used 0 = Default stack (MSP) is used If it is in the Thread or base level, the alternate stack is the PSP. There is no alternate stack for handler mode, so this bit must be zero when the processor is in handler mode.
CONTROL[0]	0 = Privileged in Thread mode 1 = User state in Thread mode If in handler mode (not Thread mode), the processor operates in privileged mode.

35

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

CORTEX M3 – OPERATION MODES

- The Cortex-M3 processor supports two modes and two privilege levels.

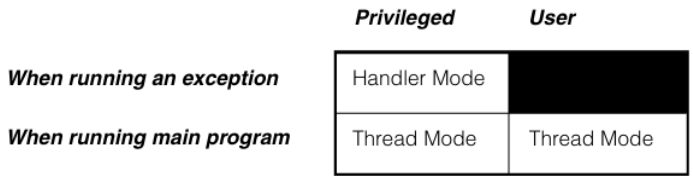


Figure 3.6 Operation Modes and Privilege Levels in Cortex-M3

- In the user access level (Thread mode), access to the System Control Space, or SCS—a part of the memory region for confi guration registers and debugging components—is blocked. Furthermore, instructions that access special registers (such as MSR, except when accessing APSR) cannot be used. If a program running at the user access level tries to access SCS or special registers, a fault exception will occur.

36

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

CORTEX M3 – OPERATION MODES

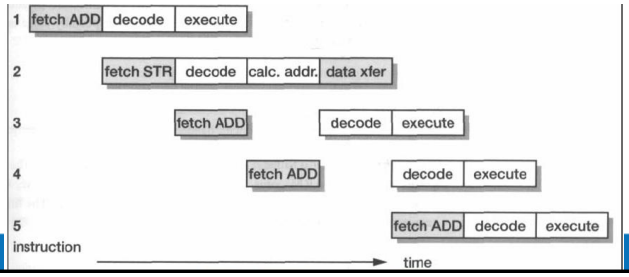
- Software in a privileged access level can switch the program into the user access level using the Control register. When an exception takes place, the processor will always switch to a privileged state and return to the previous state when exiting the exception handler.
- A user program cannot change back to the privileged state directly by writing to the Control register. It has to go through an exception handler that programs the Control register to switch the processor back into privileged access level when returning to Thread mode

37

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

INSTRUCTION CYCLE

- FETCH – DECODE – EXECUTE
- Fetch
  - The CPU retrieves the instruction from memory
  - Memory[PC] → Instruction Register
- Decode: The Control Unit decodes the instruction
- Execute: The ALU performs the operation



38

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

BYTE ORDERING

- **ARM Cortex-M uses *Little-Endian***
  - In **little-endian**, the **least significant byte (LSB)** is stored at the **lowest memory address**
- **Why Little-Endian?**
  - Cortex-M is designed for microcontrollers and embedded systems.
  - Little-endian simplifies:
    - Accessing smaller data types (byte, half-word)
    - Interfacing with typical memory systems
    - Compatibility with most compilers and toolchains
- ARM architecture *can* support big-endian, but:
  - Cortex-M3/M4/M7 typically do NOT implement big-endian mode
  - Most microcontroller vendors ship Cortex-M devices **only in little-endian**.

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

THUMB INSTRUCTION

- What is Thumb?
  - A compact **16-bit** instruction set
  - Enhanced with **Thumb-2** (mix of 16-bit & 32-bit instructions)
  - Designed for **higher code density** and **better performance** on microcontrollers
  - Cortex-M **only executes Thumb**, not ARM 32-bit instructions
- Benefits:
  - Smaller program size → saves Flash memory
  - Faster execution on systems with slow memory
  - Reduced bus bandwidth
  - Maintains most capabilities of the full ARM instruction set

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

THUMB-2 OVERVIEW

- Thumb-2 Features
  - Mixed-width instructions (16-bit + 32-bit)
  - More powerful operations than classic Thumb
  - Used in Cortex-M3/M4/M7
  - Example:

```
ADD   R0, R1, R2    ; 16-bit
ADD.W R0, R1, R2    ; 32-bit
```

- Example

```
MOV R0, #5
ADD R1, R0, #3
SUB R2, R1, #1
B   loop
BL  function
```


41

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ASSEMBLER LANGUAGE: BASIC SYNTAX

- In assembler code, the following instruction formatting is commonly used:

```
label
        opcode operand1, operand2,... ; Comments
```

- Example:

```
MOV R0, #0x12 ; Set R0 = 0x12 (hexadecimal)
MOV R1, #'A'  ; Set R1 = ASCII character A
```


42

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ASSEMBLER LANGUAGE: USE OF SUFFIXES

In assembler for ARM processors, instructions can be followed by suffixes

Table 4.1 Suffixes in Instructions

Suffix	Description
S	Update APSR (flags); for example: `ADDS R0, R1 ; this will update APSR`
EQ , NE, LT, GT, and so on	Conditional execution; EQ = Equal, NE = Not Equal, LT = Less Than, GT = Greater Than, etc. For example: `BEQ <Label> ; Branch if equal`

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ASSEMBLER LANGUAGE: USE OF SUFFIXES

- Example
  - Traditional

```
ADD  R0, R1      ; R0 = R0 + R1, using Traditional Thumb syntax
ADD  R0, R0, R1 ; Equivalent instruction using UAL syntax
```

  - UAL (Unified Assembler Language)

```
ADDS   R0, #1 ; Use 16-bit Thumb instruction by default
              ; for smaller size
ADDS.N R0, #1 ; Use 16-bit Thumb instruction (N=Narrow)
ADDS.W R0, #1 ; Use 32-bit Thumb-2 instruction (W=wide)
```

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ASSEMBLER LANGUAGE: MOVING DATA

- One of the most basic functions in a processor is transfer of data. In the Cortex-M3, data transfers can be of one of the following types:
  - Moving data between register and register
  - Moving data between memory and register
  - Moving data between special register and register
  - Moving an immediate data value into a register

```
MOV R8, R3
MVN R8, R3
```

- Example

MOV R0,#5
MOV R1,#3
MOV R2,R0
R0 = ?; R1 = ?; R2 = ?

46

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

ASSEMBLER LANGUAGE: MOVING DATA

Table 4.14 Commonly Used Memory Access Instructions

Example	Description
`LDRB Rd, [Rn, #offset]`	Read byte from memory location Rn + offset
`LDRH Rd, [Rn, #offset]`	Read half-word from memory location Rn + offset
`LDR Rd, [Rn, #offset]`	Read word from memory location Rn + offset
`LDRD Rd1,Rd2, [Rn, #offset]`	Read double word from memory location Rn + offset
`STRB Rd, [Rn, #offset]`	Store byte to memory location Rn + offset
`STRH Rd, [Rn, #offset]`	Store half word to memory location Rn + offset
`STR Rd, [Rn, #offset]`	Store word to memory location Rn + offset
`STRD Rd1,Rd2, [Rn, #offset]`	Store double word to memory location Rn + offset

47

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING
01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

Assembler Language: Processing Data

- The Cortex-M3 provides many different instructions for data processing. A few basic ones are introduced here.
- Many data operation instructions can have multiple instruction formats

```
ADD    R0, R1      ; R0 = R0+R1
ADD    R0, #0x12   ; R0 = R0 + 0x12
ADD.W  R0, R1, R2  ; R0 = R1+R2
```

- These are all ADD instructions, but they have different syntaxes and binary coding.
- When 16-bit Thumb code is used, an ADD instruction changes the fl ags in the PSR.
- However, 32-bit Thumb-2 code can either change a fl ag or keep it unchanged.

```
ADD.W  R0, R1, R2 ; Flag unchanged
ADDS.W R0, R1, R2 ; Flag change
```

48

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND EN
01, Vo Van Ngan street, Thu Duc ward, Ho Chi
hcmute.edu.vn

Assembler Language: Processing Data

Table 4.18 Examples of Arithmetic Instructions

Instruction	Operation

`ADD Rd, Rn, Rm ; Rd = Rn + Rm`
`ADD Rd, Rm ; Rd = Rd + Rm`
`ADD Rd, #immed ; Rd = Rd + #immed`
	ADD operation

`ADC Rd, Rn, Rm ; Rd = Rn + Rm + carry`
`ADC Rd, Rm ; Rd = Rd + Rm + carry`
`ADC Rd, #immed ; Rd = Rd + #immed + ; carry`
	ADD with carry

`ADDW Rd, Rn,#immed ; Rd = Rn + #immed`	ADD register with 12-bit immediate value

`SUB Rd, Rn, Rm ; Rd = Rn – Rm`
`SUB Rd, #immed ; Rd = Rd – #immed`
`SUB Rd, Rn,#immed ; Rd = Rn –#immed`
	SUBTRACT

`SBC Rd, Rm ; Rd = Rd - Rm – ; carry flag`
`SBC.W Rd, Rn, #immed ; Rd = Rn – #immed - ; carry flag`
`SBC.W Rd, Rn, Rm ; Rd = Rn – Rm – ; carry flag`
	SUBTRACT with borrow (carry)

`RSB.W Rd, Rn, #immed ; Rd = #immed –Rn`
`RSB.W Rd, Rn, Rm ; Rd = Rm - Rn`
`MUL Rd, Rm ; Rd = Rd * Rm`
`MUL.W Rd, Rn, Rm ; Rd = Rn * Rm`
	Reverse subtract
	
	Multiply

`UDIV Rd, Rn, Rm ; Rd = Rn /Rm` `SDIV Rd, Rn, Rm ; Rd = Rn /Rm`	Unsigned and signed divide

49

TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE HCMC University of Technology and Engineering

THANK YOU



66